

LANGAR — Blueprint

گروندد بیرونی: هیچ تغییری در **self-improvement** برای domain-agnostic و generic یک فریم‌وورک نمی‌شود مگر اینکه در برابر یک مرجع حقیقت بیرونی برگشت‌ناپذیر land یک سیستم خودبهبوددهنده خود فریم‌وورک است — مستقل از هر دامنه source-of-truth، اعتبارسنجی شود. این سند

ناوردای بنیادی: No self-improvement
اصل مرکزی: External Truth > Internal Consensus
lands without anchored calibration $\geq$ threshold. **Version:** 0.1 · **Status:** reference spec ·
Coupling: domain-agnostic (zero business logic)

۱. اصول و فلسفه سیستم (The Core Philosophy)

۱.۱ تعریف‌ها (Definitions)

مفهوم	تعریف دقیق
Anchoring (لنگر انداختن)	بستن یک ادعا یا تصمیم سیستم به یک رویداد بیرونی واقعی و حل‌شده (نه به نظر یک مدل). لنگر، نقطه‌ای بیرون سیستم است که سیستم نمی‌تواند آن را تولید یا جعل کند.
Ledger (دفتر) (لنگر)	تنها مرجع حقیقت. tamper-evident و append-only از حکم‌های انسانی store یک
Verdict (حکم)	`correct: قضاوت برگشت‌ناپذیر انسانی روی اینکه ادعای سیستم درست بود یا نه
Calibration (کالیبراسیون)	میزان تطابق قضاوت مرجع درون سیستمی (مثلاً پنل ارزیاب) با حکم لنگرخورده‌ی همان «روی مورد‌های حل‌شده. سنجی «آیا سیستم با واقعیت می‌خواند subject».
Gate (گیت)	halt می‌دهد یا allow اجرا می‌شود و یا land تابع تصمیمی که قبل از هر
Agreement Spiral (مارپیچ توافق)	به سرعت به هم می‌رسند و همدیگر را تأیید می‌کنند، (ها/داورها agent) حالتی که اجزای درونی در حالی که این هم‌گرایی هیچ ارتباطی با درستی ندارد. علامت مشخصه: کاهش disagreement در برابر مرجع بیرونی calibration درونی بدون افزایش.
Sycophancy	که سیستم به سمت پاسخ دلپذیر/راحت هم‌گرا می‌شود نه پاسخ درست spiral حالت خاص
Drift	جابه‌جایی تدریجی رفتار سیستم از مرجع بیرونی، حتی اگر هر قدم منطقی به نظر برسد.
System	کشش ساختاری سیستم (یا حلقه‌ی انسان+سیستم) به اعلام موفقیت/آماده‌بودن صرفاً به‌خاطر

Sunk-Cost Bias	با خروجی قبلی، به جای ارزیابی مستقل بیرونی. consistency تلاشی که خرج شده. تمایل به meta. در سطح spiral یک محرک قوی.
Tamper-Evidence	تضمین اینکه هر دستکاری گذشته آشکار می‌شود (نه اینکه غیرممکن شود). هدف واقعی tamper-proofness. سیستم همین است، نه

چرا به این سیستم نیاز داریم؟ ۱.۲

که اجزایش کار هم را نمره می‌دهند، یک نقطه ضعف ساختاری دارد: هیچ مرجعی multi-agent یک سیستم بیرون خود سیستم نیست. وقتی اجزا هم‌گرا می‌شوند، این هم‌گرایی به غلط «درست شدن» تعبیر می‌شود، چون خالص این مشکل را تشدید self-critique و reflection. هیچ چیز بیرون از حلقه نیست که تکذیبش کند. را بالا می‌برند internal consistency می‌کنند، چون فقط

واقعی یک هسته‌ی مشترک دارد و هیچ‌کدام به مقدار استدلال درونی تکیه high-performing الگوی تیم‌های نمی‌کنند:

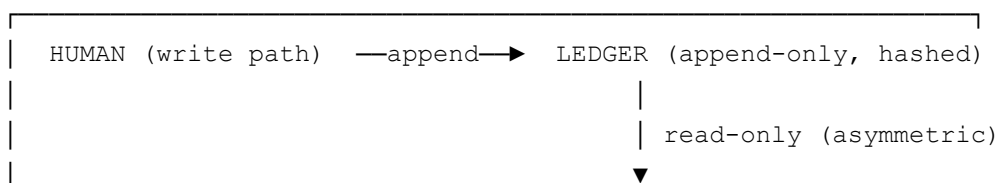
- verifiable feedback loops (نتیجه‌ی واقعی، نه نظر)
- human-in-the-loop نهایی گیت
- structured disagreement / contrarian identity
- iteration caps (سقف سخت تکرار)
- tamper-evident / auditable records
- promotion با measurable real-world lift فقط

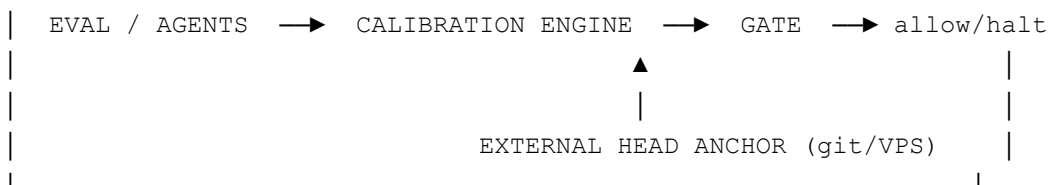
در آن‌ها outcome فقط در دامنه‌هایی امن است که self-improvement: نتیجه‌گیری فلسفی سنجش‌پذیر، یا حکم انسانی KPI نتیجه‌ی واقعی کاربر، موفقیت اجرای کد، یک باشد externally verifiable در دامنه‌های غیرقابل اعتبارسنجی، خودبهبوددهی نباید فعال شود. (ثبت‌شده‌ی بیرونی

یک چراغ سبز جعلی می‌دهد calibration، خطر مهم‌تر از نبود سیستم: اگر دفتر با داده‌ی مصنوعی/تستی پر شود mechanical liveness ≠ epistemic grounding. می‌سازد بدون ذاتش grounding — یعنی سیستم ظاهر

۲. معماری و استراکچر سیستم (System Architecture)

۲.۱ اجزای اصلی (Components)





مسئولیت	جزء
تضمین ۵ ناوردا hash حکم‌ها؛ زنجیره‌ی append-only ذخیره‌ی	Ledger
tamper-evidence. → می‌خورد hash هر رکورد به رکورد قبلی	Hash-Chain
می‌کند append فقط انسانی. تنها مسیری که	Write Path
agent/eval مسیر. eventual-consistent، فقط خواندنی، نامتقارن	Read Path
مورد اخیر یک دامنه n محاسبه‌ی تطابق پتل-در-برابر-لنجر روی	Calibration Engine
promotion. تنها نقطه‌ی land. قبل از هر allow/halt تصمیم	Gate
تشخیص جایگزینی → append به یک مخزن بیرونی بعد از هر commit_head hash	External Head Anchor
کل فایل	

۲.۲ ناوردای‌ها (Non-Negotiable Invariants)

1. **Append-only** — هیچ UPDATE / DELETE به قبلی ارجاع‌دهنده به رکورد جدید = تصحیح
2. **Human-write-only** — agentها read-only (در سطح credential/OS، فقط application نه فقط).
3. **External outcome** — هر رکورد به یک نتیجه‌ی بیرونی واقعی بسته است، نه نظرِ مدل
4. **Tamper-evidence** — هر تغییر گذشته زنجیره را از همان نقطه می‌شکند hash زنجیره‌ی
5. **Asymmetric reconciliation** — eval و هرگز نمی‌نویسد و خودش را با دفتر می‌سنجد؛ هرگز نیاز به همگامی-بلادرنگ ندارد. (به یکنوایی نیاز دارد، نه تازگی).

۲.۳ Domain-Agnostic Record Schema — ساختار دیتای خام

هر رکورد یک ردیف زنجیره‌ای است. فیلدها مستقل از دامنه‌اند:

field	type	شرح (domain-agnostic)
entry_id	uuid	شناسه‌ی یکتای رکورد
prev_hash	hex64	hash (genesis = ۶۴ صفر) رکورد قبلی

ts	ISO-8601	یک ساعت مرجع واحد.
author	string	شناسه‌ی لنگر انسانی که حکم را صادر کرده.
domain	string	که این رکورد به آن تعلق دارد verifiable دامنه‌ی
subject_ref	string	«شناسه‌ی پایدار» چیزی که قضاوت می‌شود.
claim	string	چه پیش‌بینی/پیشنهادی کرد subject سیستم برای این
outcome	string	نتیجه‌ی بیرونی واقعی (آنچه عملاً رخ داد)
verdict	enum	`correct
confidence	float[0,1]	اطمینان خود انسان.
evidence	string/uri	اشاره به شاهد بیرونی
hash	hex64	SHA256(canonical_json(body_without_hash) prev_hash) .

شده نیست **hash** است و جزو بدنه‌ی ordering فقط برای (storage شماره‌ی ترتیبی) seq

(canonical): تعریف زنجیره

```
canonical_json := JSON با کلیدهای sorted، بدون whitespace، UTF-8
hash(entry)    := SHA256( canonical_json(entry \ {hash}) || prev_hash )
genesis.prev_hash := "0" * 64
verify_chain    := mismatch = بازمحاسبه‌ی کل زنجیره؛ اولین
```

است. کلاسی بیشتر verdict agnostic فریم‌وورک نسبت به تعداد کلاسی **Verdict-space** و **chance-rate** اند. (تأثیرش در binary کمتر → تشخیص بهتر. پیش‌فرض ۳ کلاس؛ بعضی دامنه‌ها chance-agreement → ۳.۲§)

مدل تهدید (صادقانه) ۲.۴

external هر تغییر محتوا را آشکار می‌کند؛ hash را می‌بندند؛ زنجیره‌ی app-level ها نوشتن تصادفی و trigger tamper- را نمی‌گیرند — تضمین **root user** جایگزینی کل فایل را آشکار می‌کند. هیچ‌کدام جلوی یک anchor credential/OS ها باید در لایه‌ی read-only agent اجبار واقعی. tamper-proofness است نه **evidence**؛ یک لایه‌ی دفاع مضاعف است application در read-only mode باشد؛

۳. منطق گیت و محرک‌های توقف (Gate Logic & HALT Triggers)

۳.۱ HALT (هر کدام = HALT) شرط‌های

:اگر هر یک از این‌ها برقرار باشد HALT

- H1 Cold-start / Insufficient grounding : $n_overlap < min_cases$
- H2 Weak calibration : $calibration_lower_bound < threshold$
- H3 No-better-than-chance : $binomial\ null-test$ رد نشود ($p \geq \alpha$)
- H4 Agreement-spiral signal : disagreement ولی
- H5 Broken chain : $verify_chain()$ ناموفق
- H6 Stale/diverged anchor : $head \neq head$ محلی لنگر بیرونی

:فقط اگر ALLOW

فعال نباشد H1..H6 هیچ‌کدام از

مشخص برای تغییر پیشنهادی وجود داشته باشد Ledger-Anchoring Plan یک AND

صفر رکورد) escalate. cold-start و HALT، حالت امن‌پیش‌فرض است. در شک HALT: نکته‌ی طراحی می‌دهد — و این یعنی سیستم درست کار می‌کند، نه خراب HALT همیشه (واقعی

۳.۲ قلبِ منطق — (Sample-Size Awareness) آگاهی از حجم نمونه

روی نمونه‌ی کوچک ناامن است؛ نویز را به‌جای سیگنال ($raw\ rate \geq threshold$) مسئله: آستانه‌ی خام نرخ تأیید می‌کند.

احتمال عبور، (p_0 پنل مستقل از واقعیت، با نرخ تطابق شانس) زیر فرض نل **false-pass** کمی‌سازی تصادفی:

$$false_pass(n, t, p_0) = P(X \geq [t \cdot n]) \quad \text{where } X \sim \text{Binomial}(n, p_0)$$

(binary حالت محافظه‌کارانه‌ی — $p_0 = 0.5$, $raw\ threshold = 0.8$) نمونه‌ی محاسبه‌شده

n	($\geq$) لازم	false_pass
5	4	18.75%
10	8	5.47%
20	16	0.59%

تقریباً از هر ۵ پنل تصادفی، یکی چراغ سبز می‌گیرد. غیرقابل قبول $min_cases=5$, $threshold=0.8$ یعنی با →

به‌جای نرخ خام، کران پایین بازه‌ی **Lower Confidence Bound (Wilson score, one-sided)** — راه حل کوچک → بازه‌ی پهن → کران n است sample-size-aware اطمینان نرخ واقعی را آستانه بگذار. این خودبه‌خود HALT. → پایین کم

$$\hat{p} = k / n$$

2. **Pitfall detection** — spiral/sycophancy بهینه هست؟ آیا برای «بهتر به نظر رسیدن درونی» assumption کافی است یا بیشتر grounding می‌کنیم نه نتیجه بیرونی؟
3. **External grounding** — high-performers چگونه با فلسفه ledger (external > internal) می‌خوانند؟
4. **Calibration & anchoring plan** — می‌شود؟ گیت چگونه موفقیت را خواهد append دقیقاً چه رکوردی — سنجید؟ چه نتیجه واقعی‌ای آن را اثبات/ابطال می‌کند؟
5. **Decision** — فقط با مسیر. **HALT + escalate** → case بالا یا spiral ضعیف یا ریسک grounding → proposal. روشن. anchoring/calibration.

۳.۵ Iteration Cap (سقف تکرار)

throughput self-improvement: های مجاز سقف دارد. علت ساختاری land برای هر دامنه، در هر پنجره زمانی، تعداد اعتبارسنجی شده با تعداد حکم‌های واقعی انسان در واحد زمان بسته می‌شود، نه با آرزو. **improvement**. های پشت سرهم بدون گروند تازه را می‌بندد land، سقف.

۴. نقشه راه پیاده‌سازی (The Roadmap)

سخت تست‌پذیر دارد Definition of Done (DoD) هر فاز ماژولار است و یک.

Phase 0 — Frozen Kernel / Ledger Core

append , hash! زنجیره + UPDATE / DELETE های بلاک‌کننده trigger با SQLite: **باز:** verify_chain , read_prefix(since=) , head_hash ; read-only mode: external head anchor.

DoD:

- کار می‌کند append + verify_chain .
- UPDATE / DELETE بلاک است .
- می‌شوند detect هر دو relink دستکاری محتوا و .
- رد می‌شود read-only در حالت append .
- کار می‌کند divergence تشخیص + head anchor roundtrip .

Phase 1 — Interactive Ingestion (Cold-Start خروج از)

می‌کند و append ، که یک نتیجه واقعی را از انسان می‌گیرد (record.py) ساده و تعاملی CLI: **باز:** یک head . **DoD:** list و verify می‌کند! + دستور git commit را به

- شده append واقعی outcome رکورد واقعی از یک ≥ 1 .

- شده commit به صورت بیرونی head.
- می شود verify زنجیره.

Phase 2 — Calibration (مسیر خواندن)

که پنل را با حکم لنگر خورده می سنجد؛ منطق `calibration(domain, panel_verdicts, n)` بساز
DoD: گزارش (تصحیح ها جایگزین می شوند) «verdict per subject» آخرین

- شده درست محاسبه می شود `set.seed` نرخ تطابق روی یک.
- شدن تصحیح ها تست شده `supersede`.
- کار می کند `read_prefix(since=)` با `incremental` خواندن.

Phase 3 — Gate v1 (آستانه ی ثابت)

در `pre-land` ثابت؛ اتصالش به عنوان چک اجباری `min_cases + threshold` با `gate()` بساز
DoD: orchestrator.

- می دهد `HALT` پایین `calibration` روی `gate`.
- می دهد `HALT` کم `case` روی.
- می دهد `ALLOW` روی پنل کالیبره.
- کند `land` نمی تواند چیزی `ALLOW` بدون `orchestrator`.

⚠ است 4 Phase، production، برای نمونه ی کوچک ناامن است (۳.۲۹). نسخه ی `Gate v1`!

Phase 4 — Sample-Size-Aware Gate (آماري)

در برابر `binomial null-test`؛ افزودن `Wilson LB ≥ threshold` خام با `threshold` بساز؛ جایگزینی
DoD: iteration cap؛ افزودن `spiral detector (H4)` ها؛ افزودن `marginal` برآورد شده از

- یک ۵/۵ کامل کوچک را رد می کند `gate`.
- نباشد `spiral` رد شده و α و θ در سطح `LB ≥ threshold` فقط وقتی `ALLOW`.
- هدف `false-pass` طوری مستند شده اند که `α, z, min_cases`.

Phase 5 — Multi-Anchor / Scale (توزیع شده)

ادغام تاریخ های `partial order` به مثابه `reconciliation` → `log_append-only` بساز؛ چند لنگر انسانی → چند
DoD: `quorum`؛ قواعد `calibration_per-anchor`؛ (مستقل)

- می شوند `deterministic reconcile` مستقل به صورت `log` دو.
- می شود `aggregate` ها `anchor` روی `calibration`.

- مستند است consensus/quorum سیاست.

کافی است؛ این بلاکچین نیست MVP این «مسئله‌ی بعدی» است، نه حالا. تک‌لنجر برای

Operational Invariants (cross-cutting)

- شود commit به مخزن بیرونی head، append بعد از هر.
- کامل اجرا شود audit به صورت دوره‌ای به عنوان verify_chain().
- پذیرفته و مستند باشد (سقف حکم‌های انسانی/هفته) throughput ceiling.

record.py (Phase 1, generic) قالب — A ضمیمه

```
#!/usr/bin/env python3
"""Generic interactive ingester for the Langar ledger (Phase 1)."""
import argparse
from langar import Ledger, write_head_anchor, git_commit_head

def cmd_add(led, anchor, repo):
    print("یک حکم روی یک نتیجه‌ی واقعی حل‌شده ثبت کن")
    e = led.append(
        author=input("author: ").strip(),
        domain=input("domain: ").strip(),
        subject_ref=input("subject_ref: ").strip(),
        claim=input("claim (سیستم چه پیش‌بینی کرد): ").strip(),
        outcome=input("outcome (واقعاً چه شد): ").strip(),
        verdict=input("verdict [correct|wrong|partial]: ").strip(),
        confidence=float(input("confidence [0..1]: ").strip()),
        evidence=input("evidence (uri/note): ").strip(),
    )
    write_head_anchor(led, anchor)
    git_commit_head(repo, anchor, f"anchor {e['subject_ref']} -> {e['hash'][:12]}")
    print(f"appended {e['entry_id']} hash={e['hash'][:12]}...")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("cmd", choices=["add", "verify", "list"])
    ap.add_argument("--db", default="langar.db")
    ap.add_argument("--anchor", default="langar.head")
    ap.add_argument("--repo", default=".")
    a = ap.parse_args()
    led = Ledger(a.db)
    if a.cmd == "add":
        cmd_add(led, a.anchor, a.repo)
```

```

elif a.cmd == "verify":
    ok, bp, msg = led.verify_chain()
    print("OK" if ok else f"BROKEN at {bp}: {msg}")
elif a.cmd == "list":
    for r in led.read_prefix():
        print(f"{r['seq']:>4} {r['domain']:<16} {r['subject_ref']:<20} "
              f"{r['verdict']:<8} {r['hash'][:10]}...")
    led.close()

if __name__ == "__main__":
    main()

```

B — Default Config (قابل override per-domain) ضمیمه

```

verdict_classes: [correct, wrong, partial]
gate:
    min_cases: 20          # کران سخت پیش‌فرض (نه 5)
    threshold: 0.80        # نه نرخ خام Wilson LB، روی
    z: 1.645               # 95% one-sided
    alpha: 0.05            # binomial null-test
    p0: estimate           # نه marginal از
    iteration_cap_per_window: 1
spiral:
    window: 10
    rule: "slope(disagreement) < 0 AND slope(calibration_LB) <= 0 -> HALT"
anchor:
    commit_head_after_every_append: true
    full_verify_audit: periodic

```

اند و باید روی داده واقعی هر دامنه کالیبره شوند. آنچه بادوام است خود ناوردا [Speculative] این اعداد نمی‌شود land لنگر بالای آستانه calibration ی بدون self-improvement است، نه اعداد: هیچ